

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering

Academic Year 2025-2026

Rubrics for Lab Experiments

Class: B.E. Computer
Engineering

Subject Name: SMA Lab

Semster: VIII

Subject Code: CSDL8023

Practical No.	8
Title:	Implement content based and collaborative based filtering
Date of Performance:	3-12-2025
Roll No.	9913
Student	Mark Lopes

Evaluation

Performance Indicator	Below Average	Average	Good	Excellent	Marks
On Time Submission (2)	Not submitted(0)	Submitted after deadline (1)	Early or on time submission (2)	—	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Signature of the Teacher:

Aim: Implement content based and collaborative based filtering

Code:

./main.py

```
"""
Movie Recommendation System - Core Logic
Content-Based and Collaborative Filtering Implementation
"""

import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# =====
# CONTENT-BASED FILTERING
# =====

def calculate_content_similarity(movies_df):
    genres_text = movies_df["genres"].fillna("")

    # Convert genres to TF-IDF matrix
    tfidf = TfidfVectorizer(token_pattern=r"[A-Za-z]+")
    tfidf_matrix = tfidf.fit_transform(genres_text)

    # Calculate cosine similarity between all movies
    cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

    return cosine_sim

def get_content_recommendations(movie_title, movies_df, cosine_sim, top_n=10):
    # Find the movie in the dataframe
    movie_match = movies_df[movies_df["title"] == movie_title]

    if movie_match.empty:
        return pd.DataFrame()

    # Get the index of the movie
    idx = movie_match.index[0]

    # Get similarity scores for this movie with all other movies
    sim_scores = list(enumerate(cosine_sim[idx]))

    # Sort by similarity score (descending), skip first one (the movie itself)
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)[1 : top_n + 1]

    # Extract movie indices and similarity scores
    movie_indices = [i[0] for i in sim_scores]
    similarity_scores = [i[1] for i in sim_scores]

    # Create recommendations dataframe
    recs = movies_df.iloc[movie_indices][["title", "genres"]].copy()
    recs["Similarity Score"] = [round(s, 4) for s in similarity_scores]
```

```

recs.reset_index(drop=True, inplace=True)
recs.index = range(1, len(recs) + 1)

return recs

# =====
# COLLABORATIVE FILTERING
# =====

def calculate_user_similarity(ratings_df):
    # Create user-item matrix (rows: users, columns: movies, values: ratings)
    user_item_matrix = ratings_df.pivot_table(
        index="userId",
        columns="movieId",
        values="rating"
    ).fillna(0.0)

    # Calculate cosine similarity between users
    user_similarity = cosine_similarity(user_item_matrix)

    # Convert to DataFrame for easier indexing
    user_similarity_df = pd.DataFrame(
        user_similarity,
        index=user_item_matrix.index,
        columns=user_item_matrix.index
    )

    return user_similarity_df, user_item_matrix

def get_collaborative_recommendations(
    user_id,
    ratings_df,
    movies_df,
    user_similarity_df,
    user_item_matrix,
    top_n=10
):
    """
    User-based collaborative filtering using similarity-weighted averages.

    Algorithm:
    1. Find users similar to the target user
    2. For each movie not rated by target user:
        - Calculate weighted average of ratings from similar users
        - Weight = similarity between target user and other user
    3. Return top-N movies with highest predicted ratings

    Args:
        user_id: ID of the user to get recommendations for
        ratings_df: DataFrame with ratings data
        movies_df: DataFrame with movies data
        user_similarity_df: Precomputed user similarity matrix
        user_item_matrix: User-item ratings matrix
        top_n: Number of recommendations to return

    Returns:
        DataFrame with recommended movies, genres, and predicted ratings
    """

```

```

"""
# Check if user exists
if user_id not in user_similarity_df.index:
    return pd.DataFrame()

# Get similarity vector for this user (similarity with all other users)
sim_vector = user_similarity_df.loc[user_id].copy()

# Set self-similarity to 0 (don't consider own ratings)
sim_vector.loc[user_id] = 0.0

# Reshape for matrix multiplication
sim_values = sim_vector.values.reshape(1, -1)      # Shape: (1, num_users)
ratings_matrix = user_item_matrix.values           # Shape: (num_users, num_movies)

# Calculate weighted sum of ratings
# Each movie's score = sum of (similarity × rating) from all users
weighted_sum = sim_values @ ratings_matrix          # Shape: (1, num_movies)

# Sum of absolute similarity weights (for normalization)
sum_of_weights = np.abs(sim_values).sum(axis=1, keepdims=True) + 1e-8

# Calculate predicted ratings
pred_ratings = (weighted_sum / sum_of_weights).flatten() # Shape: (num_movies,)

# Create Series with predictions indexed by movieId
movie_ids = user_item_matrix.columns.to_numpy()
pred_series = pd.Series(pred_ratings, index=movie_ids)

# Remove movies already rated by this user
user_ratings_row = user_item_matrix.loc[user_id]
already_rated = user_ratings_row[user_ratings_row > 0].index
pred_series = pred_series.drop(index=already_rated, errors="ignore")

if pred_series.empty:
    return pd.DataFrame()

# Get top-N movies with highest predicted ratings
top_movies = pred_series.sort_values(ascending=False).head(top_n)

# Join with movie titles and genres
rec_df = movies_df[movies_df["movieId"].isin(top_movies.index)][
    ["movieId", "title", "genres"]
].copy()

# Add predicted ratings
rec_df["Predicted Rating"] = rec_df["movieId"].map(top_movies)

# Sort by predicted rating
rec_df = rec_df.sort_values("Predicted Rating", ascending=False)

# Clean up
rec_df = rec_df.drop("movieId", axis=1)
rec_df.reset_index(drop=True, inplace=True)
rec_df.index = range(1, len(rec_df) + 1)

return rec_df

```

```

if __name__ == "__main__":
    # Sample data
    movies_data = {
        'movieId': [1, 2, 3, 4, 5, 6],
        'title': ['The Dark Knight', 'Inception', 'Interstellar',
                  'The Avengers', 'Iron Man', 'The Matrix'],
        'genres': ['Action|Crime|Drama', 'Action|Sci-Fi|Thriller', 'Adventure|Drama|Sci-Fi',
                   'Action|Adventure|Sci-Fi', 'Action|Adventure|Sci-Fi', 'Action|Sci-Fi']
    }

    ratings_data = {
        'userId': [1, 1, 1, 2, 2, 2, 3, 3, 3],
        'movieId': [1, 2, 3, 1, 4, 5, 2, 3, 6],
        'rating': [5, 5, 4, 4, 5, 5, 5, 5, 4]
    }

    movies_df = pd.DataFrame(movies_data)
    ratings_df = pd.DataFrame(ratings_data)

    print("=" * 60)
    print("CONTENT-BASED FILTERING")
    print("=" * 60)

    # Calculate similarity
    cosine_sim = calculate_content_similarity(movies_df)

    # Get recommendations for a movie
    movie_title = "Inception"
    recommendations = get_content_recommendations(movie_title, movies_df, cosine_sim,
top_n=3)

    print(f"\nRecommendations for '{movie_title}':")
    print(recommendations)

    print("\n" + "=" * 60)
    print("COLLABORATIVE FILTERING")
    print("=" * 60)

    # Calculate user similarity
    user_similarity_df, user_item_matrix = calculate_user_similarity(ratings_df)

    # Get recommendations for a user
    user_id = 1
    user_recs = get_collaborative_recommendations(
        user_id,
        ratings_df,
        movies_df,
        user_similarity_df,
        user_item_matrix,
        top_n=3
    )

    print(f"\nRecommendations for User {user_id}:")
    print(user_recs)

    print("\n" + "=" * 60)

```

Output:

Configuration

Select Data Source:

Sample Dataset

Local CSV Files

Local Files

Movies file path

movies.csv

Ratings file path

ratings.csv

Load Local Dataset

MovieLens Dataset
Format: movies.csv (movioid, title, genres), ratings.csv (userid, movioid, rating[, timestamp])

Deploy

Top 20 Recommendations for User 64

	Movie Title	Genres	Predicted Rating
1	X-Men: The Last Stand (2006)	Action Sci-Fi Thriller	★ 0.256
2	Clue (1985)	Comedy Crime Mystery Thriller	★ 0.210
3	I Heart Huckabees (2004)	Comedy	★ 0.203
4	Romy and Michele's High School Reunion (1997)	Comedy	★ 0.183
5	Anniversary Party, The (2001)	Drama	★ 0.183
6	Windtalkers (2002)	Action Drama War	★ 0.163
7	Home Alone (1990)	Children Comedy	★ 0.160
8	Rounders (1998)	Drama	★ 0.137
9	365 Days: This Day (2022)	Drama Romance	★ 0.136
10	The Mummy (2017)	Action Adventure Fantasy Horror Thriller	★ 0.091
11	Getaway, The (1994)	Action Adventure Crime Drama Romance Thriller	★ 0.070

Predicted Ratings Visualization

Predicted Ratings for Unwatched Movies

X-Men: The Last Stand (2006)

Clue (1985)

I Heart Huckabees (2004)

Anniversary Party, The (2001)

Predicted Rating

0.25

Configuration

Select Data Source:

Sample Dataset

Local CSV Files

Local Files

Movies file path

movies.csv

Ratings file path

ratings.csv

Load Local Dataset

MovieLens Dataset
Format: movies.csv (movioid, title, genres), ratings.csv (userid, movioid, rating[, timestamp])

Deploy

20 Sicko (2007) Documentary|Drama

Predicted Rating

★ 0.000

Predicted Ratings Visualization

Predicted Ratings for Unwatched Movies

X-Men: The Last Stand (2006)

Clue (1985)

I Heart Huckabees (2004)

Anniversary Party, The (2001)

Romy and Michele's High School Reunion (1997)

Windtalkers (2002)

Home Alone (1990)

Rounders (1998)

365 Days: This Day (2022)

The Mummy (2017)

Getaway, The (1994)

Police Academy 4: Citizens on Patrol (1987)

Critters (1986)

Wallace & Gromit in The Curse of the Were-Rabbit (2005)

Sicko (2007)

Death Proof (2007)

Fantastic Four: Rise of the Silver Surfer (2007)

Hairspray (2007)

Knocked Up (2007)

Away from Her (2006)

Predicted Rating

0 0.05 0.1 0.15 0.2 0.25

Social Media Analytics Lab

Fr. Conceicao Rodrigues College of Engineering

Experiment No 8

Shrink dataset size

Local file loading fix

Fix UI display issues - Claude

Collaborative filtering fix

Movie Recommendation Sys

localhost:8502

Coolers - The sup...Adobe Acrobat

Deploy

Configuration

Select Data Source: ⓘ

Sample Dataset

Local CSV Files

Local Files

Movies file path

movies.csv

Ratings file path

ratings.csv

Load Local Dataset

MovieLens Dataset

Format: movies.csv (movieId, title, genres), ratings.csv (userId, movieId, rating[, timestamp])

Dataset Overview

Total Movies5,410

Total Ratings20,000

Total Users17,118

Avg Rating3.55★

Content-Based

Collaborative

Hybrid

Surprise

Content-Based Filtering

Get movie recommendations based on genre similarity

Select a movie: Jumanji (1995)

Number of recommendations: 20

Get Recommendations

Top 20 Recommendations for: Jumanji (1995)

	Movie Title	Genres	Similarity Score
1	Indian in the Cupboard, The (1995)	Adventure Children Fantasy	1.0000
2	NeverEnding Story III, The (1994)	Adventure Children Fantasy	1.0000
3	Return to Oz (1985)	Adventure Children Fantasy	1.0000
4	NeverEnding Story, The (1984)	Adventure Children Fantasy	1.0000

Experiment No 8

Shrink dataset size

Local file loading fix

Fix UI display issues - Claude

Collaborative filtering fix

Movie Recommendation Sys

localhost:8502

Coolers - The sup...Adobe Acrobat

Deploy

Configuration

Select Data Source: ⓘ

Sample Dataset

Local CSV Files

Local Files

Movies file path

movies.csv

Ratings file path

ratings.csv

Load Local Dataset

MovieLens Dataset

Format: movies.csv (movieId, title, genres), ratings.csv (userId, movieId, rating[, timestamp])

Similarity Scores Visualization

Content Similarity Scores

Movie Title

The Chronicles of Narnia: The Lion, the Witch and the Wardrobe (1988)

Alice Through the Looking Glass (2016)

Chronicles of Narnia: The Voyage of the Dawn Treader, The (2010)

Chronicles of Narnia: Prince Caspian, The (2008)

Golden Compass, The (2007)

Bridge to Terabithia (2007)

Chronicles of Narnia: The Lion, the Witch and the Wardrobe, The (2005)

Harry Potter and the Sorcerer's Stone (a.k.a. Harry Potter and the Philosopher's Stone) (2001)

Santa Claus: The Movie (1985)

NeverEnding Story, The (1984)

Return to Oz (1985)

NeverEnding Story III, The (1994)

Indian in the Cupboard, The (1995)

MirrorMask (2005)

Return to Halloweentown (2006)

Night at the Museum: Secret of the Tomb (2014)

Son of the Mask (2005)

Lemony Snicket's A Series of Unfortunate Events (2004)

Borrowers, The (1997)

Warriors of Virtue (1997)

Similarity Score

1

0.98

0.96

0.94

0.92

0

0.2

0.4

0.6

0.8

1

Similarity Score

Social Media Analytics Lab

